Plectis: Bounded Public Evidence from a Private AI-Built System

What a stranger can check, what I can only report, and what remains unestablished

Will Cook

July 2026

Abstract

Plectis is a public software repository built out of a larger private system that cannot be published. I treat it here as a case study in a general problem: what may a stranger reasonably conclude from a curated set of small, runnable public fragments taken, or adapted, from a private body of work they cannot see? Each of the 88 public components joins a claim to a fixed input, a command, stored output records, an explicit pass rule, and a stated limit. The count is an inventory, not a score. The public repository lets a reader test claims about these published files and their outputs. It cannot, by itself, establish the private system's history, the representativeness of the selection, or the correctness of success criteria the project supplies for itself. I developed the private system over about ten months by directing AI coding agents; that account is reported, not proved, and the paper's argument does not depend on it. The paper defines the object in ordinary terms, examines one component from input to limit, sets out four distinctions that govern what runs can and cannot show, and states what stronger evidence would require. A dated record of a verification pass at a named commit is given in an appendix.

1 The problem

Plectis is a public software repository, stored at github.com/wcook04/plectis. A *repository* is an organised project folder, usually kept with its revision history; to *clone* a repository is to copy that folder and history to another computer. Everything this paper says a stranger can check is in that repository.

The repository exists because of a difficulty that is not specific to me. I have a larger private software system that I cannot responsibly publish: it contains personal records, credentials, live browser and account information, my working notes, and unfinished tools. I would still like some of its claims to be answerable to strangers, rather than resting entirely on my say-so. The question this paper examines is what such answerability can amount to: what may a stranger reasonably conclude from a curated set of runnable public fragments taken, or adapted, from a system they cannot see? Plectis is my working answer. This paper describes that answer and measures how far it reaches.

I am 22 and study economics. Over about ten months I built the private system by directing AI coding agents: I set its objectives, rules, and review criteria, while the agents were the principal implementers, writing, running, testing, and revising the software under my direction. No other person worked on it. That is all *AI-built* means in this paper. It is testimony about origin, stated once and not relied on afterwards: Plectis can expose the resulting public files and runs, but it cannot independently prove that account of how the private work was produced. What

the account does explain is why the publication problem arose, and why every published choice described below is my responsibility rather than the agents'. The artefact does not prove the story, and the evidence below would be exactly as strong, and exactly as limited, if every line had been written by hand.

The boundary of the evidence can be stated before any command is run:

Status	**What belongs in it**
Publicly checkable	The contents of a named public commit, its registered commands, the files those commands write, and whether its public tests pass.
Reported here	The private system's history, the role of AI agents in producing it, and the private origin of published material.
Not established here	Whether the public selection represents the private whole, whether the mechanisms generalise beyond their fixed cases, whether self-supplied success criteria are independently correct, and whether the private system works reliably at all.

Reading this paper requires no programming. Reproducing its runs requires a terminal (the text window in which commands are typed), Git (the standard program for copying and versioning repositories), Python 3.11 or later, the `pip` installer, and basic command-line use. So that the body of the paper can stay with the argument, the exact commands, environment details, and installation notes are collected in the appendix, which is a dated record of one verification pass at the commit named there.

2 The component contract

A *component* in Plectis is one registered, testable part of the project. Its *fixture* is a fixed sample input, and its *receipt* is a saved, machine-readable record of a reference run. A *validator* is the program that applies a component's pass and refusal rules. A *commit* is a named, saved version of the repository. These terms describe files and commands; they do not grade the importance of what any component does. (Inside the repository a component is called an *organ*, and the code package is named `microcosm_core`: the project was previously called Microcosm, and several file and command names keep the old name. The generated page `ORGANS.md`, which gives a plain description and a first command for every component, is named after the first.)

At commit `57ffb7f5830c` the registry, the machine-readable list of components, contains 88 entries. Every entry is required to connect the same things: a claim stated in prose; a fixture; a command; the output files the command writes; a stored receipt from a reference run; an explicit rule for what counts as a pass and what must be refused; and a stated limit on what a pass establishes.

An assertion about private software can only be believed or doubted as testimony. A component converts the assertion into a procedure, and a procedure can fail in front of a stranger. The conversion does not make the claim true. What it changes is the form disagreement can take: a reader who doubts a component's claim can point at a particular input, a particular line of the validator, or a particular number in the output, instead of arguing with my description of software they cannot see.

A component may also end in a declared refusal rather than an answer. The worked example below refuses an input format it does not support. Components that depend on an external tool, such as the Lean proof checker (a program that verifies mathematical proofs), must record a bounded refusal when the tool is absent, rather than reporting success. *Runnable* therefore

means that the operation and its declared failure boundary are present in public form; it does not mean that every optional dependency exists on every machine. A refusal is informative only when it is declared in advance and distinguishable from a breakdown: a predeclared refusal marks the edge of what the component claims, while an unexpected crash is a defect. The two must not be read as the same kind of event.

3 One run, examined

The component `batch8_audio_level_rms_port` computes a loudness level from short arrays of audio samples. I chose it for the worked example because it is ordinary. It needs no mathematics beyond a square root and no knowledge of the private system, and the claim it makes is small. Its value here is that every part of the evidence can be seen at once.

The reported origin is this. The private system contains a small macOS recording application, and one of that application's functions reduces a buffer of audio samples to a level between zero and one. The public component re-expresses that calculation in Python. The component's code and its stored receipts name the private file it claims to re-express (`AudioLevelMonitor.swift`); a stranger cannot open that file, so the name is a report, not evidence. What the stranger can check is everything else.

The declared calculation is the standard one called root-mean-square (the `rms` in the component's name): square each sample, average the squares, take the square root, then multiply by eight and cap the result at one. Sixteen-bit integer samples are first divided by 32,767 to bring them onto the same scale. An empty input must produce zero. An unrecognised format name must be refused rather than guessed at.

The component's public command runs it against its fixture and writes fresh output outside the repository:

```
PYTHONPATH=src python3 -m \
  microcosm_core.organs.batch8_audio_level_rms_port run \
  --input fixtures/first_wave/batch8_audio_level_rms_port/input \
  --out /tmp/plectis-audio-rms \
  --acceptance-out /tmp/plectis-audio-rms-acceptance.json
```

At commit `57ffb7f5830c`, the run produced:

Case	Input	Expected	Observed	Result
Decimal samples	0.0, 0.05, −0.05, 0.1	0.489897949	0.489897949	pass
16-bit integer samples	0, 1638, −1638, 3277	0.489892970	0.489892970	pass
Values above the cap	1.0, −1.0, 0.5	1.000000000	1.000000000	pass
Empty input	(none)	0	0	pass
Unknown format	`pcm24`	refusal	refusal	pass

Because the inputs and the rule are both printed above, the first row can be checked without the repository or a computer: the squares of the four samples average to 0.00375, whose square root is 0.061237…, and eight times that is 0.489897… A reader with a calculator can recompute this expected value from the stated formula alone. For this one case the reader can act as their own *oracle*, the general name for whatever determines what an answer should have been, independently of the program under test.

Almost nothing else in the collection is like that, which is why the example matters. Elsewhere the project supplies the implementation, the input, the expected values, and the validator, all from the same hand. A test assembled that way can pass for four different reasons: the implementation and the expectation are both correct; the two share a mistake; the chosen cases happen to sit where the implementation behaves; or the validator checks a weaker property than the prose claim suggests. A matching result therefore establishes repeatability under that public test. Independent correctness needs an oracle that does not depend on me, as elementary arithmetic does for the first row, or fresh inputs whose expected results I never chose.

The refusal row records a designed boundary, not a failure: an input naming an unsupported encoding (`pcm24`, a format this component does not handle) must be rejected with an error, and is. The stored reference for the whole table is `batch8_audio_level_rms_port_validation_receipt.json` under `receipts/first_wave/batch8_audio_level_rms_port/`, so a fresh run is compared against a fixed record rather than against memory. A stored record by itself does not prove that the stated command produced it; its use is that anyone can generate a fresh one and compare the two.

The limit is part of the component. This run checks one public Python calculation over synthetic sample arrays (arrays composed for the test, not captured from a device). It does not run the original application, start an audio session, request microphone permission, or capture sound, and it cannot establish that the named private file is where the calculation came from. Nor does the table show that the task was difficult (it was not), that the component is useful elsewhere, or that the remaining entries resemble it. What it shows is that one assertion now has a form in which a stranger can rerun it, dispute its rule, perturb its input, or replace my expected value with a better-derived one.

4 Four distinctions

Interpreting the collection needs four distinctions. Each separates something a stranger can observe from something the observation may seem to imply, and each names the further evidence that would be needed to cross the gap. The limits attached to individual components are instances of these.

Public execution versus private provenance

A reader can establish what a named public commit contains and what its commands do. No public run establishes where the material came from. Plectis keeps records at that boundary: copied files carry public import records naming their claimed sources, together with fingerprints (a *fingerprint* is a short value computed from a file's contents, chosen so that any change to the contents changes the value). Matching fingerprints show that a public file agrees with the project's public record of it. But I control both sides of that comparison, so agreement is evidence of internal consistency, not of origin. The same holds for the audio component's named source file: the name is an entry I wrote. Origin claims could only be strengthened by something a public repository cannot supply about itself, such as commitments published before the fact, or a trusted person permitted to examine both sides. Until then, provenance is reported.

Repeatability versus correctness

A run that reproduces the stored receipt shows that the public code, on the supplied input, in a sufficiently similar environment, produces the same result again. Correctness is a different property: it needs a standard for what the result should have been, and a standard is only worth something if it does not merely restate the implementation. The worked example makes the difference concrete. Its first row has an oracle independent of the project, namely elementary arithmetic; most components have only the project's own expected values and validator. Passing tests anywhere in Plectis should be read at that strength and no more.

Selected cases versus general behaviour

This distinction applies twice. Within a component, the fixture's cases are a few points in a large space of possible inputs, and behaviour elsewhere is untested. Across the collection, the published components are a selection from a private whole, and the selection was mine. I chose which mechanisms could be given public form, which inputs to freeze, which expected values to store, and which pass rules to enforce. Nothing in the public record shows how many candidates there were, what was excluded beyond the privacy rules, or whether the collection resembles ordinary work in the private system. A collection can consist entirely of genuine items and still give an unrepresentative picture of what it was drawn from; a display case does exactly this whenever the curator keeps the failures in a drawer.

The registry's count should be read in that light. 88 entries defines the published population exactly, which is what a later evaluation would need in order to sample from it and report failure rates against a fixed denominator. The count is an inventory, not a score, and even the unit of counting is a choice I control, since one mechanism could have been registered as several components, or several as one.

Risk reduction versus guarantee

The publication process runs checks whose honest description is that they reduce specific risks. Automated scans search copied material for specified patterns of credentials, personal information, and private file paths, and record what was omitted or rejected; a scan establishes that the scanner did not find the patterns it was designed to find, and nothing stronger. Pages such as the component map are generated from the registry, and tests compare page and registry, which prevents the particular failure of prose drifting away from records; a generator and its records can still be wrong together. Plectis is also designed not to read the private repository at all, and that design is itself public and testable. None of this amounts to a guarantee that no secret, no identifying detail, and no inconsistency survives. Publication remains a judgement about acceptable risk, and I made it.

The provenance and risk-reduction distinctions pull against each other in a way worth noticing. Establishing provenance would require someone to examine private material, and the privacy constraint that motivates the whole design forbids exactly that examination in public. Some claims are therefore permanently in the reported category, short of a confidential review. The honest treatment is to label them, not to dress them up as public proof through bookkeeping that one maintainer controls.

5 What the collection contains

At commit `57ffb7f5830c` the 88 components fall into 7 groups:

Group	Count	What its components do
Getting started	2	Show a new reader where to begin and guide a first inspection.
Project mapping	12	Map files and written rules, find relevant material, and route a question to the part of the project that owns it.
Computer-checked mathematics	20	Prepare, repair, or check bounded steps used in formal mathematical work.
AI reliability and safety tests	20	Replay fixed cases involving hostile instructions, software boundaries, stored information, tool use, and benchmark rules.
Research-method tests	9	Exercise fixed cases involving replication, conflicting forecasts, explaining model behaviour, simulation, and laboratory safety.
Public-copy maintenance	20	Check copied source, generated pages, publication boundaries, and disagreement between records and derived files.
Work recovery	5	Record unfinished changes and test how work resumes after interruption.

The groups do not share a subject; computer-checked mathematics has nothing in particular to do with audio levels or with recovering interrupted work. Their unity is the contract of Section 2: these are the places in the private system where a mechanism could be given a bounded, runnable public form. The collection is organised by how its claims are checked, not by subject.

The registry also records how each public form was produced. Four routes cover all 88 entries, and each is a different trade between fidelity to the private original and what a stranger can check:

Route	Count	What it preserves	What it cannot show
Record-checked copied source	21	The exact text of non-secret source files, checked against a public import record.	That the recorded origin is real; record and file share one maintainer.
Public reimplementation	27	The behaviour of a bounded calculation, exercised on fixed cases.	Any relation to the private original beyond my report; the original text is absent.
Direct or external-tool run	22	A local computation, or a named external tool's result, captured with its context.	Behaviour where the tool or environment differs; the result is as strong as the tool and the case.
Contract checker	18	Whether a public input satisfies a declared structural rule.	Whether the declared rule is the right rule to require.

Copied source preserves the most text and demonstrates the least behaviour. A reimplementation demonstrates behaviour while surrendering textual identity; the worked example is one of the 27 in that route. An external-tool run borrows the tool's authority along with the tool's limits. A contract checker is worth exactly the contract its author chose to require. A reader who wants to know which trade a given component made can read its registry entry, which names its operation, command, receipt paths, evidence route, and limit.

6 What stronger evidence would look like

A passing run shows that named public code produced a stated result from a supplied input in a stated environment. Applying Section 4: the runs reported here do not establish private provenance, representativeness, correctness beyond self-supplied criteria, or anything about usefulness, security, originality, or the reliability of the private system as a whole. The computer-checked mathematics components prove no more than their named public fixtures, and no other repository's size or subject matter is used as evidence here. This note does not report an independent, randomly selected audit over fresh inputs; nothing of that kind has happened yet.

What would move each boundary is known, and the stages are ordered by who controls the consequential choices.

Independent repetition comes first: a person with no involvement in the project reruns the supplied commands in their own environment and publishes what happened, including failures. That would remove the dependence on my machine and my tacit knowledge. It would say nothing about correctness.

Evaluator-controlled selection comes next: the evaluator chooses components from the full registry, without substitution when one proves awkward, and original failures stay on the record even if I later repair them. That addresses curation within the public collection. It cannot address whether the collection represents the private system, because the private pool stays invisible.

Fresh inputs with independently derived expectations move repeatability towards correctness: inputs chosen after a version is frozen, and expected results worked out through some route that does not pass through my code, whether independent calculation, a second implementation, or an external reference. Bounded correctness claims for the evaluated cases would then be available; general correctness still would not.

Comparison on outside tasks is what a usefulness claim would need: someone bringing a task that was not designed around the project's own categories, and comparing the component against an ordinary alternative, which for several of these components would be a short conventional script. Usefulness is comparative, and nothing in this paper compares.

A provenance review is the only stage that could bear on the origin story: a trusted person examining selected private material against the public collection under confidentiality. The privacy cost may reasonably never be paid. If it is not, the origin story remains testimony, and this paper is written so that its argument does not depend on it.

Each stage moves one class of consequential choice out of my hands. That, rather than more components or more records, is what stronger evidence means for an object like this.

7 What to inspect

The useful first review is small. Clone the repository, run the guided first inspection (`plectis tour`, shown with its exact commands in the appendix), and pick one claim you doubt, rather than the component most likely to impress. Read the component's description and stated limit before running anything. Run its command on the supplied input and compare the fresh output with the stored receipt. Then read the validator and ask whether its pass rule bears on the prose claim; perturb the input where practical; and keep whatever fails, because a discrepancy is a finding, and `CONTRIBUTING.md` in the repository sets out how to report one. One successful run supports one bounded conclusion. Broader conclusions wait on the evidence of Section 6,

and most of it does not yet exist.

A A dated reproduction record

Everything in this appendix is an observation about one commit, made on 17 July 2026 in one environment: macOS 15.6.1 (arm64) with Python 3.13.7. The commit's full identifier is

`57ffb7f5830cc446a2cbd8083d5d80bcab2af563`

abbreviated `57ffb7f5830c` elsewhere in the paper. None of this appendix is a permanent property of the project. Later commits will change file counts, and other machines can behave differently; where that matters, the body of the paper does not depend on it.

Names. The repository is github.com/wcook04/plectis. As Section 2 notes, the code package inside it is named `microcosm_core`. The installed command is `plectis`, and an older command name, `microcosm`, still works as an alias.

Getting the code and running without installing.

```
git clone https://github.com/wcook04/plectis
cd plectis
PYTHONPATH=src python3 -m plectis tour --format text .
```

To reproduce the exact state this appendix describes, run `git checkout 57ffb7f5830c` after `cd plectis`; without it the commands run against the newest version instead.

In a clean clone at this commit, the tour read 5,018 project files, found five possible reading orders for them, selected the one that starts from the README (`readme_onboarding_route`), wrote 21 files under a new `.microcosm/` directory beside the project, and reported the examined source files unchanged. Its output names three handles carried by every recorded observation: the evidence behind it, the event record it came from, and the limit on what it authorises. The `.microcosm/` directory can be deleted and generated again. The repository's README shows the same command reporting a different file count from a different day. Both numbers are what they claim to be, dated outputs of one run each, and neither is a stable measurement of the project.

Installing.

```
python3 -m pip install .
plectis tour --format text .
```

The installed package declares no third-party runtime dependencies. The build step requires a sufficiently recent `setuptools`, which a cold machine may download when `pip` builds the package. After installation, the core component runs make no network or AI-model calls. Before anything is installed, `./bootstrap.sh` checks a fresh clone and reports whether the shipped inputs and declared limits are intact.

The worked example. The command in Section 3 writes its fresh output under `/tmp/plectis-audio-rms`. The tracked reference to compare it against is the receipt named in Section 3, under `receipts/first_wave/batch8_audio_level_rms_port/`. At this commit the run reported every case in the Section 3 table as a pass, with observed values equal to the stored ones.

Project-wide checks. At this commit, `make check` loads the component registry and finishes in under a second, and `python3 scripts/check_lean_proof_trust.py` scans the 58 Lean proof files shipped in the repository for constructs that would let a file look proved without being proved: placeholder proofs, axioms the project granted itself, evaluation that bypasses the checker's core, declarations marked unsafe or partial, and raised limits on how much the checker examines. Both passed. (The 58 files are a different unit of counting from the 20 computer-checked mathematics components, so the two numbers are not comparable.) `make ci` adds installation, the public test suite, and two end-to-end example runs; it is the same floor the repository's continuous-integration service runs. These commands test named properties of the public checkout. They do not certify the private system.

The full public test suite at this commit was not green: three of its tests failed, one because the repository's front-door profile check did not classify this paper's own files at the repository root, and two in documentation- and record-consistency checks that predate this revision. The profile defect is repaired in a later commit. The failures are recorded here rather than smoothed over, which is what Section 7 asks a reader to do with their own failed runs.

The paper itself. A script, `scripts/check_public_system_paper.py`, recomputes every count used in this paper from the public registries, verifies that the named commit exists, verifies that the worked example and its receipt are the ones the registry declares, and fails if the paper and the repository disagree. The public test suite runs it, so drift between this document and the records it describes is a test failure. That makes the paper one of the repository's checked documents, and the check should be read with Section 4 in mind: agreement between a document and records with one maintainer is internal consistency, not external truth.